Home Assignment 1

Group 20: Anton Blomgren, Aleksi Löf

April 27, 2026

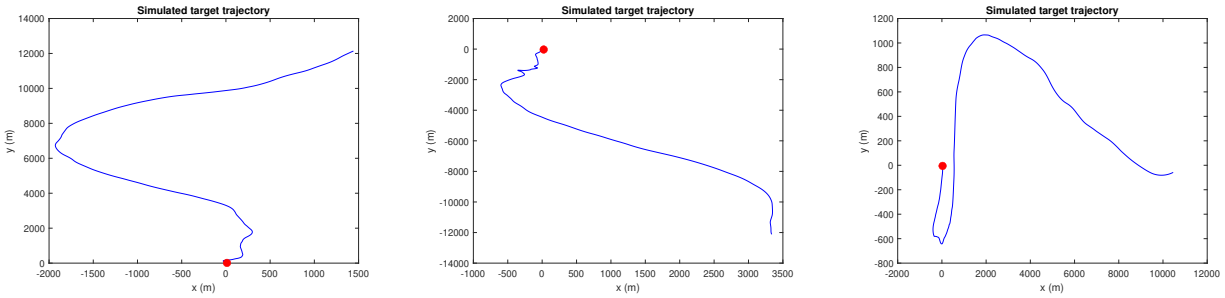
1 Problem 1

$\{\mathbf{X}_n\}_{n\in\mathbb{N}}$ is not a Markov chain since $\mathbf{X}_{n+1}$ depends on both $\mathbf{X}_n$ and $\mathbf{Z}_n$, this violates the markov property

$$P(X_{n+1} \mid X_0, X_1, \ldots, X_n) \neq P(X_{n+1} \mid X_n).$$

If $\tilde{\mathbf{X}}_n = \left(\mathbf{X}_n^\top, \mathbf{Z}_n^\top\right)^\top$ this means that we have knowledge of both $\mathbf{Z}_n$ and $\mathbf{X}_n$. Thus $\{\tilde{\mathbf{X}}_n\}_{n\in\mathbb{N}}$ is a Markov chain.

$$P(\tilde{X}_{n+1} \mid \tilde{X}_0, \tilde{X}_1, \ldots, \tilde{X}_n) = P(\tilde{X}_{n+1} \mid \tilde{X}_n)$$



(a) Trajectory 1 (b) Trajectory 2 (c) Trajectory 3

Figure 1: Simulated target trajectories with $m = 500$ steps.

The randomized paths in Figure 1 look reasonable for a trajectory of a moving vehicle.

2 Problem 2

$\{(\tilde{\mathbf{X}}_n, \mathbf{Y}_n)\}_{n\in\mathbb{N}}$ forms a *hidden Markov model* (HMM) for two reasons. First, as concluded in Problem 1, $\{\tilde{\mathbf{X}}_n\}_{n\in\mathbb{N}}$ is a Markov chain, here a hidden state process where the true state $\tilde{\mathbf{X}}_n$ is never directly observed. Second, the observation $\mathbf{Y}_n$ depends on the hidden state only through the current value $\tilde{\mathbf{X}}_n$, since each Y_n^ℓ is a deterministic function of the position (X_n^1, X_n^2) disturbed by some noise V_n^ℓ. In particular, given $\tilde{\mathbf{X}}_n$, the observation $\mathbf{Y}_n$ is conditionally independent of all past and future states and observations. This is the HMM structure.

The *RSSI* that the mobile unit receives from the ℓth basis station at time $n \in \mathbb{N}$ is modeled by

$$Y_n^\ell = v - 10\eta \log_{10} \left\| \left(X_n^1, X_n^2\right)^\top - \boldsymbol{\pi}_\ell \right\| + V_n^\ell.$$

To find the transition density $p(y_n \mid \tilde{x}_n)$ of $Y_n \mid \tilde{X}_n$, we note that each noise term V_n^ℓ is independent and identically distributed as $\mathcal{N}(0, \varsigma^2)$. Conditioning on $\tilde{x}_n$ fixes the position (x_n^1, x_n^2), rendering the distance term deterministic. We therefore define the conditional mean for each base station ℓ as

$$\mu_\ell = v - 10\eta \log_{10} \left\| \left(x_n^1, x_n^2\right)^\top - \boldsymbol{\pi}_\ell \right\|,$$

so that $Y_n^\ell \mid \tilde{x}_n \sim \mathcal{N}(\mu_\ell, \varsigma^2)$ for each $\ell = 1, \ldots, s$. This because adding a constant to a Gaussian just shifts the mean. Since the noise variables V_n^ℓ are independent across stations, the joint conditional density can be calculated as

$$p(y_n \mid \tilde{x}_n) = \prod_{\ell=1}^{s} \frac{1}{\sqrt{2\pi}\,\varsigma} \exp\left(-\frac{(y_n^\ell - \mu_\ell)^2}{2\varsigma^2} \right),$$

In the implementation for this project the log-likelihood is used instead for numerical stability, thus the logarithm yields

$$\ln p(y_n \mid \tilde{x}_n) = \sum_{\ell=1}^{s} \left(-\frac{(y_n^\ell - \mu_\ell)^2}{2\varsigma^2} - \log\left(\sqrt{2\pi}\,\varsigma\right) \right).$$

3 Problem 3

To get an estimate $\{(\hat{\tau}_n^1, \hat{\tau}_n^2)\}_{n=0}^m$ of the true trajectory, we want to sample recursively from the smoothing distribution given in the assignment.

$$f(\tilde{\mathbf{x}}_{0:n} \mid \mathbf{y}_{0:n}) = \frac{f(\tilde{\mathbf{x}}_{0:n}, \mathbf{y}_{0:n})}{f(\mathbf{y}_{0:n})} = \frac{q(\tilde{\mathbf{x}}_0)p(\mathbf{y}_0 \mid \tilde{\mathbf{x}}_0)\prod_{k=1}^n p(\mathbf{y}_k \mid \tilde{\mathbf{x}}_k)q(\tilde{\mathbf{x}}_k \mid \tilde{\mathbf{x}}_{k-1})}{f(\mathbf{y}_{0:n})}$$

But since the denominator $f(\mathbf{y}_{0:n})$ is intractable, we instead draw from the proposal and correct with importance weights. Choosing the prior transition density as proposal, i.e. $q(\tilde{\mathbf{x}}_k \mid \tilde{\mathbf{x}}_{k-1}) = p(\tilde{\mathbf{x}}_k \mid \tilde{\mathbf{x}}_{k-1})$, the unnormalized importance weights are

$$\omega(\tilde{\mathbf{x}}_{0:n}) = \frac{f(\tilde{\mathbf{x}}_{0:n}, \mathbf{y}_{0:n})}{q(\tilde{\mathbf{x}}_{0:n})} = \frac{q(\tilde{\mathbf{x}}_0)p(\mathbf{y}_0 \mid \tilde{\mathbf{x}}_0)\prod_{k=1}^n p(\mathbf{y}_k \mid \tilde{\mathbf{x}}_k)\, p(\tilde{\mathbf{x}}_k \mid \tilde{\mathbf{x}}_{k-1})}{q(\tilde{\mathbf{x}}_0)\prod_{k=1}^n p(\tilde{\mathbf{x}}_k \mid \tilde{\mathbf{x}}_{k-1})} = \prod_{k=0}^n p(\mathbf{y}_k \mid \tilde{\mathbf{x}}_k),$$

where the transition densities cancel exactly. This yields the recursive update

$$\omega_{n+1}^i = \omega_n^i \cdot p(\mathbf{y}_{n+1} \mid \tilde{\mathbf{x}}_{n+1}^i)$$

After proper initialization, particles $\tilde{X}_0^i = (X_0^i, Z_0^i)$ are drawn from the prior,where $X_0^i \sim \mathcal{N}(\mathbf{0}, \mathrm{diag}(500, 5, 5, 200, 5, 5))$ and Z_0^i is drawn uniformly from $\mathbf{U}\{1, \ldots, 5\}$. The SIS algorithm then proceeds iteratively for $n = 1, \ldots, m$ as follows. Each particle is propagated jointly by drawing $\mathbf{W}_{n+1}^i \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$ and updating the state for each particle via

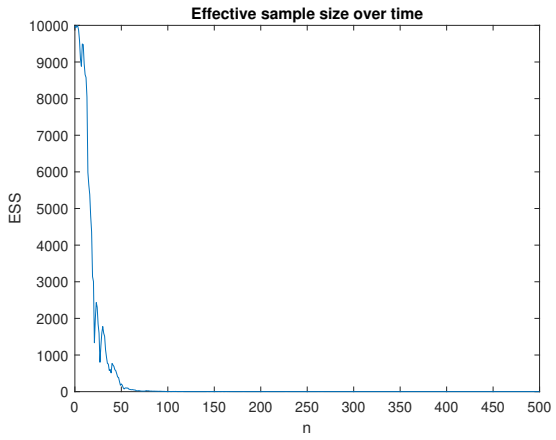
$$\mathbf{X}_{n+1} = \mathbf{\Phi}\mathbf{X}_n + \mathbf{\Psi}_z\mathbf{Z}_n + \mathbf{\Psi}_w\mathbf{W}_{n+1}$$

The driving command Z_{n+1}^i is then drawn from row $P_{Z_n^i,\cdot}$ of the transition matrix P, i.e. the next command is sampled according to the transition probabilities corresponding to the current command Z_n^i. Then we update the weights with the following relation

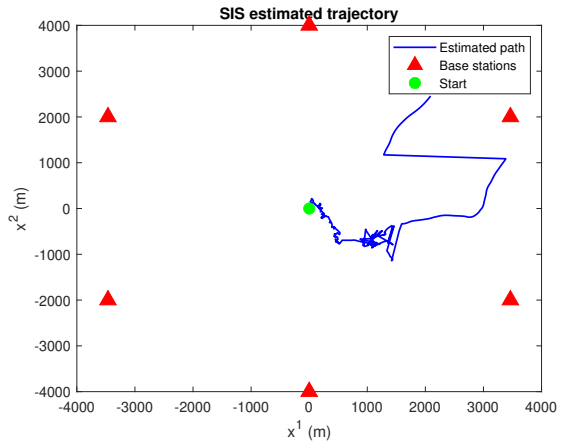
$$\ln \omega_{n+1}^i = \ln \omega_n^i + \ln p(\mathbf{y}_{n+1} \mid \tilde{\mathbf{x}}_{n+1}^i),$$

where $\ln p(\mathbf{y}_{n+1} \mid \tilde{\mathbf{x}}_{n+1}^i)$ is the log-likelihood derived in Problem 2 , computed in practice by the function `ln_obs_prob` for all N particles simultaneously with no loop over i. For numerical stability, $\max_i \ln \omega_n^i$ is subtracted before exponentiation prior to normalization, i.e. $\tilde{\omega}_n^i = \exp(\ln \omega_n^i - \max_j \ln \omega_n^j)$, after which the weights are normalized to sum up to one. The position estimates for each particle at time n are then computed as the weighted average:

$$\hat{\tau}_n^1 = \sum_{i=1}^N \frac{\tilde{\omega}_n^i}{\Omega_n} X_n^{1,i}, \quad \hat{\tau}_n^2 = \sum_{i=1}^N \frac{\tilde{\omega}_n^i}{\Omega_n} X_n^{4,i}$$



(a) Effective sample size for SIS



(b) Trajectory for SIS

Figure 2: Results for SIS

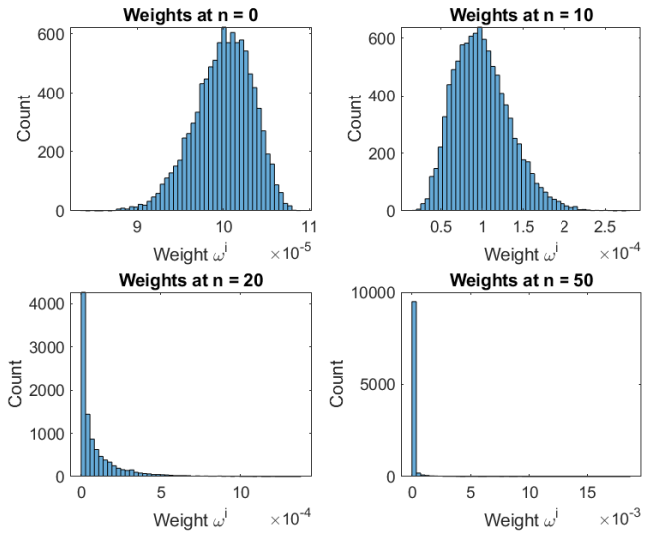


Figure 3: Weights for SIS

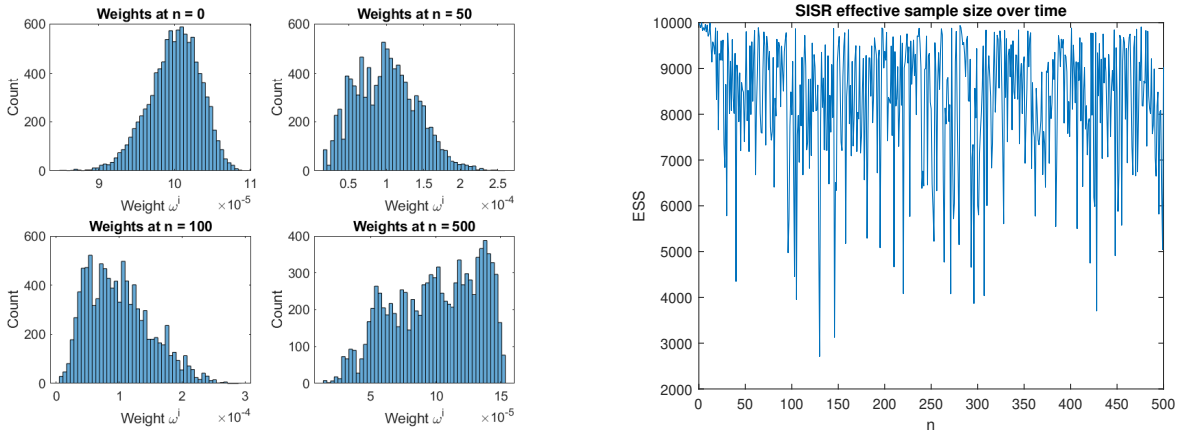
From Figure 2 (a) it is clear that after ≈ 50 iterations the number of effective particle paths collapeses to only one. This is further confirmed in Figure 3, where weight degeneracy is observed, this is consistent with theory for the SIS algorithm. This occurs since the particle weights are generated through subsequent multiplications, as mentioned in Lecture 6. One can also observe that in Figure 2 (b) after a certain time the trajectory changes behaviour. This can be explained by the fact that one particle takes all the weight after some time.

4 Problem 4

To avoid the issue of weight degeneracy we now want to use the SISR algorithm, where we at each step n perform the SIS algorithm to first update the weights, and after this implement a selection step where the particles are resampled proportional to their weights, effectively removing low-weighted particles and replicating high-weighted particles. The weight values are then reset to be uniform for the next step. This effectively mitigates the issue of weight degeneracy since the weight product never accumulates across steps, preventing any single particle from dominating the ensemble over time. Since weights are reset to $1/N$ after each resampling step, the weight update simplifies to

$$\omega_{n+1}^{i} \propto p(\mathbf{y}_{n+1} \mid \tilde{\mathbf{x}}_{n+1}^{i}),$$

In MATLAB this is done via `randsample(N, N, true, w)`, which draws N indices with replacement according to the normalized weights `w`, after which both the state matrix `X` and driving commands `Z_idx` are reindexed accordingly.

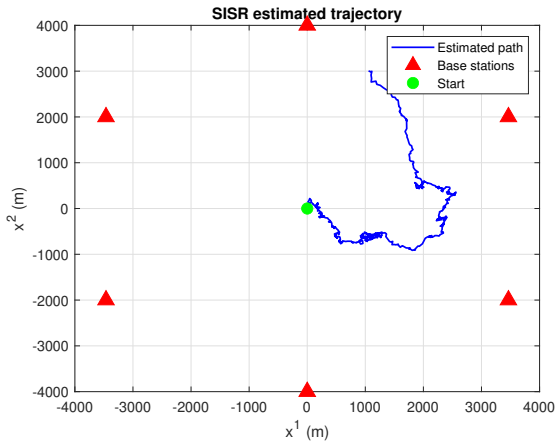


(a) Weights for SISR

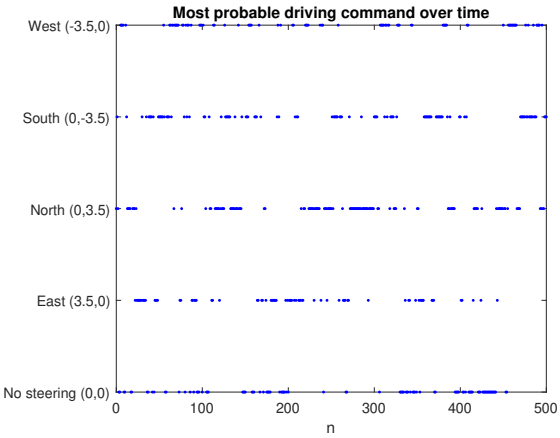
(b) Effective sample size for SISR

Figure 4: Weights and ESS for SISR

In Figure 4 we can now see that the weights have no problem of degeneracy since the weights are now well spread out for larger n, following the theory for SISR accordingly.



(a) Trajectory for SISR



(b) Most probable driving command for SISR

Figure 5: Trajectory and driving command for SISR

In Figure 5 we see the final trajectory when using the SISR algorithm. We can see that the overall path is clearly visible, in comparison to the trajectory when using the SIS algorithm in Figure 2 which became unstable after a certain time due to the weight degeneracy.

The most probable driving commands can also be seen for each time step n in Figure 5 which seems to reflect the path well also.

5 Problem 5

In this problem the observation noise standard deviation ς is unknown and must be estimated. We maximize the normalized log-likelihood

$$\ell_m(\varsigma) = \frac{1}{m} \ln L_m(\varsigma) = \frac{1}{m} \ln f_\varsigma(\mathbf{y}_{0:m}),$$

Since $f_\varsigma(\mathbf{y}_{0:m})$ is intractable the SISR algorithm is used to get an estimate by computing at each step the mean likelihood increment $\frac{1}{N} \sum_{i=1}^{N} p_\varsigma(\mathbf{y}_n \mid \tilde{\mathbf{x}}_n^i)$, giving the log-likelihood estimate

$$\ell_m^N(\varsigma) = \frac{1}{m} \sum_{n=0}^{m} \ln \left(\frac{1}{N} \sum_{i=1}^{N} p_\varsigma(\mathbf{y}_n \mid \tilde{\mathbf{x}}_n^i) \right).$$

We evaluate $\ell_m^N(\varsigma)$ over a grid $\{\varsigma_j\} = \{0.1, 0.2, \ldots, 2.9\} \subset (0, 3)$ by running the SISR algorithm once per grid point. The approximate MLE is then

$$\hat{\varsigma}_m = \arg\max_{\varsigma_j} \ell_m^N(\varsigma_j).$$

The resulting estimate is $\hat{\varsigma}_m = 2.2$, given by the maximum log-likelihood in Figure 6, and the estimated trajectory under this value is shown in Figure 7.

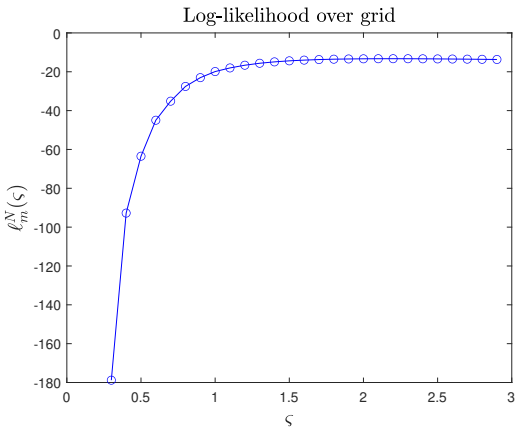


Figure 6: Log-likelihood over grid

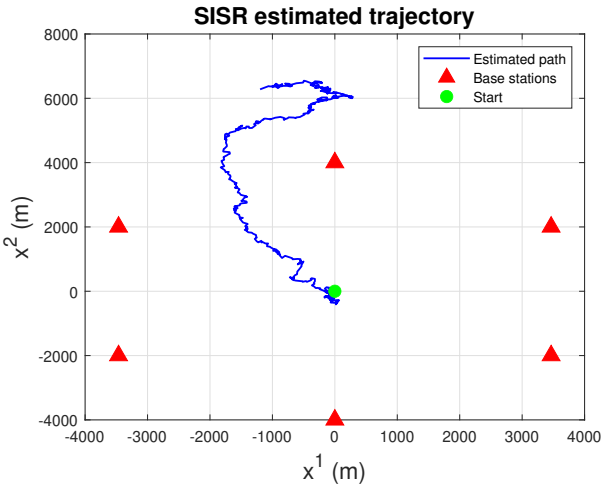


Figure 7: SISR trajectory with best ς